

SLO 1: Language Theory Concepts

1. What is Type Checking? Demonstrate Static vs Dynamic Typing Using Java

Type Checking

Type checking is the process of checking whether a variable or expression is used with the correct data type.

For example:

```
int x = 10;
```

Here, x is declared as an int, so it should store an integer value.

Type checking helps detect errors when incompatible data types are used.

Static Typing

Java is a statically typed language. In Java, the data type of a variable is known and checked at compile time.

Example:

```
class StaticExample {  
    public static void main(String[] args) {  
        int x = 10;  
        System.out.println(x);  
    }  
}
```

Here, x is declared as an int. Assigning "Hello" to x produces a compile-time error.

```
int x = 10;
```

↓

Type is checked during compilation

↓

Correct

Dynamic Typing

In a dynamically typed language, the type of a variable is determined or checked mainly at run time.

For example, Python is dynamically typed:

```
x = 10
```

```
x = "Hello"
```

The same variable can refer to values of different types at different times.

Java does not use dynamic typing for ordinary variables.

Difference

Conclusion: Java is a statically typed language because type checking is primarily performed during compilation.

2. Write a Java Program Showing Variable Scope

Variable scope refers to the region of a program where a variable can be accessed.

Java mainly demonstrates scope through local variables, block variables, instance variables, and class variables.

Example Program

```
class ScopeExample {  
    int instanceVar = 100;  
  
    static int classVar = 200;  
  
    public static void main(String[] args) {  
  
        int localVar = 10;  
  
        System.out.println("Local variable: " + localVar);  
        System.out.println("Class variable: " + classVar);  

```

```
ScopeExample obj = new ScopeExample();  
    System.out.println("Instance variable: " + obj.instanceVar);  
    if (localVar > 5) {  
        int blockVar = 50;  
        System.out.println("Block variable: " + blockVar);  
    }  
  
}  
  
}
```

Types of Scope Demonstrated

1. Local Variable

```
int localVar = 10;
```

It can be accessed only inside the method or block where it is declared.

2. Block Variable

```
if (localVar > 5) {  
    int blockVar = 50;  
}
```

blockVar can be accessed only inside the if block.

3. Instance Variable

```
int instanceVar = 100;
```

It belongs to an object and can be accessed through an object.

4. Class Variable

```
static int classVar = 200;
```

It belongs to the class and can be accessed without creating an object.

Example of Invalid Scope

```
if (true) {  
    int x = 10;
```

```
}  
  
    System.out.println(x); // Error
```

The variable x is available only inside the if block.

3. Explain the Difference Between Binding Time and Run-Time with Code Examples

Binding Time

Binding means associating a property, value, or variable with something in a program.

Binding time refers to the point in the program's life cycle when this association is made.

Binding can happen at different times, such as:

Compile time

Run time

Compile-Time Binding

When a binding is determined during compilation, it is called compile-time binding.

For example:

```
    class BindingExample {  
        public static void main(String[] args) {  
            int x = 10;  
            System.out.println(x);  
        }  
    }
```

Here, the compiler knows that:

```
    int x
```

means x is an integer variable.

The type of x is therefore bound at compile time.

Run-Time Binding

When the method to be executed is determined during program execution, it is called run-time binding or dynamic method dispatch.

Example:

```
    class Animal {  
        void sound() {  
            System.out.println("Animal sound");  
        }  
    }  
  
    class Dog extends Animal {  
        void sound() {  
            System.out.println("Dog barks");  
        }  
    }
```

```

}

class BindingDemo {
    public static void main(String[] args) {

        Animal a = new Dog();

        a.sound();
    }
}

```

Output

```

Dog barks

Although a is declared as an Animal reference:

Animal a = new Dog();

```

the object created at run time is a Dog.

Therefore, Java determines at run time that the Dog version of sound() should be executed.

Difference Between Binding Time and Run Time

Simple Representation

Compile Time

↓

Type of variable is checked

↓

Program is compiled

↓

Run Time

↓

Actual object/method behavior is determined

Static Typing | Dynamic Typing

Type is checked at compile time | Type is checked mainly at run time

Variable type is explicitly declared in Java | Type can be inferred from the value

Errors can be detected before execution | Some type errors appear during execution

Example: Java, C, C++ | Example: Python, JavaScript

Binding Time | Run Time

Association is determined before program execution | Decision is made during program execution

Commonly occurs during compilation | Occurs while the program is running

Example: variable type in Java | Example: overridden method selection

Helps detect errors early | Provides dynamic behavior

Example: `int x = 10;` | Example: `Animal a = new Dog();`

